

W4

BLOCK 1 · CLASS 4 · PRACTICAL

Rules, judge, loop.

Last week you watched a feature get built. Today you build one alone.

What we'll do.

- 1 **Check-in** — feature 1, merged.
- 2 **What an agent actually is** — and how it got that way.
- 3 **Three standards** — a rulebook, a judge, a loop.
- 4 **Feature 2, live** — we watch it build, then we check its work.
- 5 **Your turn** — one bug, one feature. Thirty-two minutes.
- 6 **Retro** — turn today's worst moment into one rule.

Show me your GitHub page.

- A branch for feature 1.
- A merged pull request.
- Not there yet? Do it first today. Everything else still works.

A chat model writes. An agent **acts**.

A CHAT MODEL

Text in, text out.

It has never seen your folder and cannot run anything. *You* copy code in, *you* paste it out, *you* find out whether it worked.

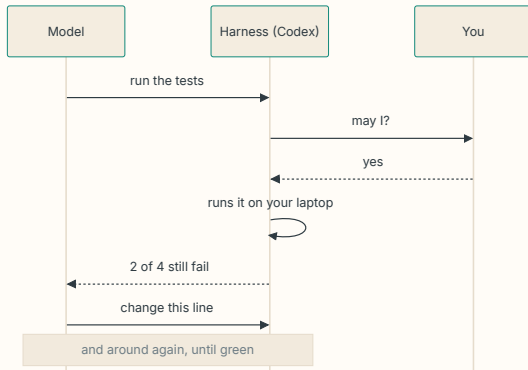
AN AGENT

The same model, with hands.

It reads your files, runs commands, edits code — and sees the result of each action before choosing the next. It stops when the job is done, or to ask you.

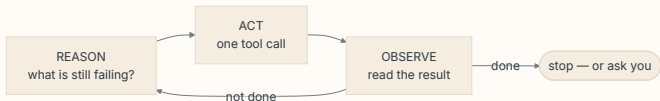
Agent = model + tools + a loop.

The model asks. The harness does.



Each arrow from the model is a **tool call** — read a file, edit a file, run a command. That is the whole vocabulary.

The loop has a name: **ReAct**.



THE HARNESS

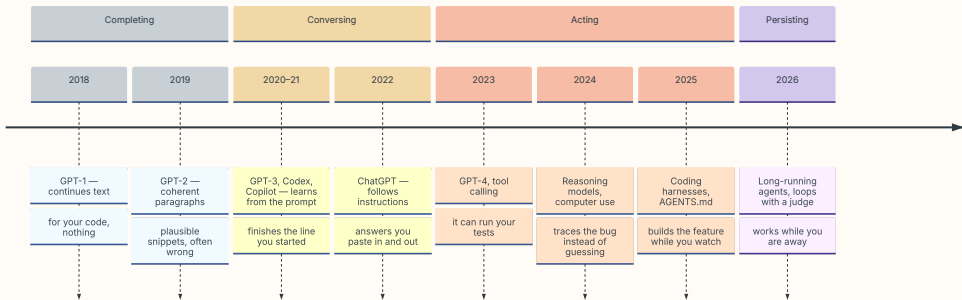
The model can only *ask*. The software that runs the tool, asks your permission, feeds the result back and decides when to stop is the **harness** — Codex, Claude Code, Cursor.

WHY YOU CARE

Every one of them runs this same loop. Learn the loop once and the tools are interchangeable.

Yao, Zhao, Yu, Du, Shafran, Narasimhan & Cao (2022). *ReAct: Synergizing Reasoning and Acting in Language Models*. ICLR 2023 · arxiv.org/abs/2210.03629

Eight years, four kinds of capability.



The point. The job changed twice: when the model learned to take **instructions**, and when it got **tools and a loop**. A model that can act for hours is why the next three slides exist.

`AGENTS.md` — the file it reads first.

WHAT IT IS

A README for agents

Codex reads it at the start of every session, before anything else. So do the other tools. Your repo has had one since Class 2.

WHAT GOES IN

What it cannot guess

The commands. The boundaries. How you want it to report back.

WHAT STAYS OUT

What it can read

Explanations of your own code. Tutorials. "Write clean code." Every extra line hides the ones that matter.

It is code: it lives in git, it gets reviewed, it gets pruned. When the agent misbehaves, the fix is usually one line here — not a longer prompt.

Three rules. One commit.

BOUNDARY

One branch per spec

Never commit straight to
`main`.

EVIDENCE

Show me the test output

The output is the proof.
Not a sentence saying so.

SACRED

Never touch the tests

If a test looks wrong, say
so and stop.

Add them to `AGENTS.md`, read the diff, commit, push. The exact wording is on the Week 4 page — copy it.

Decide what **done** means. First.

SOMETHING THAT CANNOT LIE

An agent stops when the work *looks* done. Give it a check that answers pass or fail — the failing tests are your to-do list, and green is the finish line.

NEVER MOVE THE GOALPOSTS

An agent that edits a test to make it pass has cheated, not finished.

EVIDENCE, NOT CLAIMS

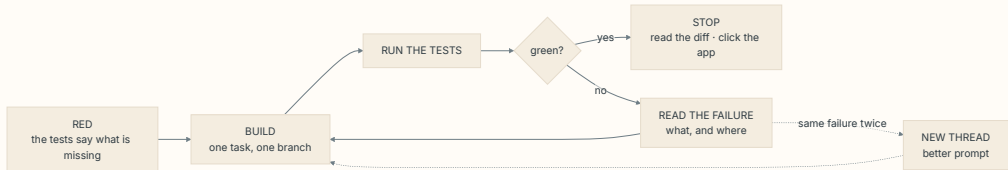
"All tests pass" is a sentence. The test output is evidence. Ask for it every time.

GREEN IS NOT THE END

The judge only checks what it was told to check. Then you read the diff and click the app. App wrong, tests green? You just found a test that should exist.

STANDARD 3 · THE LOOP

Red → **build** → **run** → **fix** → **green.**



ONE TASK PER LOOP

One bug, or one spec.
Not “fix everything you see”.

LET IT LOOP

It runs the check, reads the failure, tries again.
You read the result — you are not the messenger.

KNOW WHEN TO STOP

Green, or twice round on the same failure. Then:
new thread, better prompt.

Every good prompt has four parts.

1 · THE TASK

One thing.

Name the single spec or bug you want built. Nothing else.

2 · THE BOUNDARY

Where it may go.

This branch. And never the tests.

3 · THE JUDGE

How it knows it is done.

Run the tests after every change, and keep going until they pass.

4 · THE EVIDENCE

What to show you.

The final test output, and every file it touched.

The prompt written out in full is on the Week 4 page. Copy it and change two names.

Watch it work. Then check its work.

- 1 **Branch.** Name it after the spec.
- 2 **Red first.** See exactly what is missing before anything is written.
- 3 **New thread. Paste the prompt.** Then sit on your hands.
- 4 **Watch it loop.** Edit, run, read the failure, edit again.
- 5 **Green. Now don't trust it.** Read the diff: did it change only what the task needed?
- 6 **Click the app.** The part no test covers is the part only you can catch.
- 7 **Commit, push, pull request, merge.**

One bug on `main`. One feature on a branch.

SPLIT IT

Bug 2, then feature 2

Removing a member crashes the page.
Then expense categories.

TINY CRM

Bug 3, then feature 2

"Won this month" is always zero. Then
conversion by source.



Files, test names and the prompt: Week 4 page.

Five things you will see.

- 1 **It edited a test.** Discard it. Say the rule out loud. Run again.
- 2 **"Done" — but the tests are red.** Ask for the output. The output is the judge.
- 3 **The same failure, three times.** Stop. Read it yourself. New thread.
- 4 **It fixed three other things too.** Discard them. One task per loop.
- 5 **Green, but the app is wrong.** The judge missed something. Write down what.

Every line here is a candidate rule for your `AGENTS.md`. Keep the list.

What did it get wrong? Write the rule.

- The worst thing the agent did today. One sentence each.
- Turn it into a rule: short, specific, and you could tell whether it was followed.
- Add it to `AGENTS.md`. Say *why* in the commit message. Push.
- Three groups read theirs out.

That is how a rulebook grows — from what went wrong, one line at a time. Not from a longer prompt.

Next week you build alone.

- Finish feature 2. Green, merged, pushed.
- Your rule is in `AGENTS.md`, with the reason in the commit message.
- Push before you close the laptop.

Rulebook · judge · loop.

Block 2 is those three, on a spec of your own.